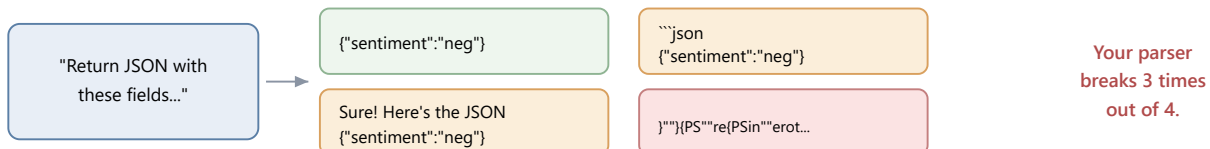


Structured Outputs and Constrained Decoding

When an LLM feeds a real system, you don't want prose. You want JSON, a function call, SQL, or code — something a program can parse.

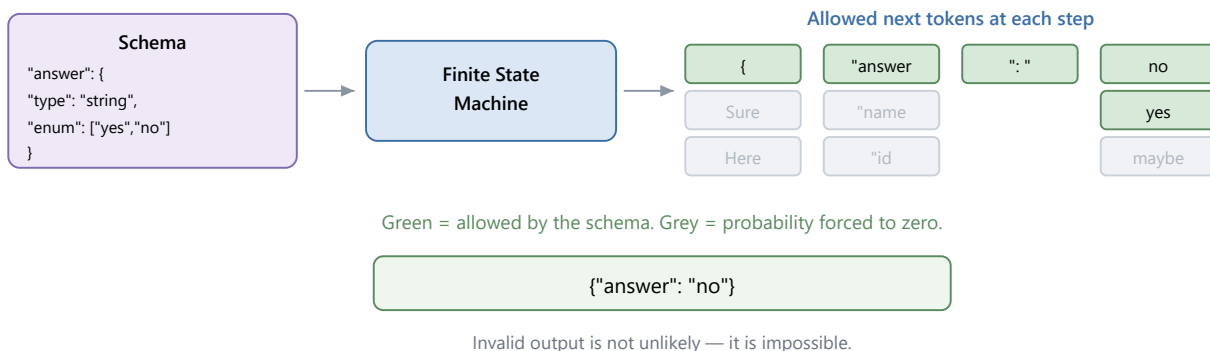
Asking politely doesn't work reliably. The same prompt, three times, gives you: valid JSON; JSON wrapped in a chatty preamble; JSON inside a markdown fence; and occasionally a sampling accident that produces mangled braces. Your parser fails on three out of four.



Prompting for structure gives you structure most of the time. Most of the time is not good enough for a pipeline.

Constrained decoding

The fix is not a better prompt. It's to stop the model from being *able* to produce invalid output. You give the runtime a schema; the runtime compiles the schema into a state machine; at every step it masks out every token that would break the schema, and the model samples only from what's left.



Constrained decoding turns your schema into a filter over the vocabulary at every single step.

The gain is dramatic. In OpenAI's own benchmark on a deliberately complex schema, compliance went from under 40% for an older model relying on prompting alone to 100% for a newer model with structured outputs enabled (OpenAI, *Introducing structured outputs in the API*). Read that as a vendor comparison of one model pair on one hard schema rather than a general law — the number you'd see depends on your model and how gnarly your schema is. The shape of the result is what generalizes: constraining the decoder converts "usually valid" into "always valid."